

Scalable Behaviour Cloning on Browser Using via Skill Distillation

Internet users collectively perform an enormous range of skilled work through web browsers, from software development and document editing to search, forms, and enterprise workflows. This activity is a highly scalable but under-exploited source of reusable browser skills. We study scalable behavior cloning for browser agents via distributed skill summarization, converting user interaction trajectories into compact natural-language skills that agents can read, retrieve, reuse, and compose directly. We show that simple trajectory summarization alone can improve benchmark performance through behavior cloning, and further introduce a distributed framework for scalable skill collection and reuse across users and tasks. This suggests that the scalability of browser agents may come less from manually designed tasks and more from the collective skills already expressed by internet users. Our project is available at: xxx.

Date: June 21, 2026



1 Introduction

Web browsers have become the universal execution interface for modern digital work. Developers move between GitHub, continuous integration systems, and documentation; operators update dashboards and enterprise workflows; researchers search, organize, and transfer information across online tools; and everyday users complete tasks in email, calendars, shopping sites, forms, and software-as-a-service applications. A browser agent that can reliably use this interface would therefore provide a general execution layer for digital labor. Early work such as WebShop showed how language agents could act in grounded web environments (Yao et al., 2023), while Mind2Web broadened the problem toward generalist web agents over real websites (Deng et al., 2023). More recent benchmarks, including WebArena and VisualWebArena, make the challenge increasingly realistic by requiring long-horizon interaction, multimodal observation, element grounding, and task-level evaluation (Zhou et al., 2024; Koh et al., 2024). WorkArena and BrowserGym further extend this line toward common knowledge-work tasks and standardized research infrastructure (Drouin et al., 2024; Chezelles et al., 2025).

At the same time, browser agents have access to a potentially scalable source of supervision: interaction traces produced by humans and agents during real web tasks. These traces encode more than clicks. Mind2Web demonstrates that demonstrations over real websites can supervise action grounding and next-step prediction (Deng et al., 2023); WebLINX adds multi-turn, dialogue-conditioned navigation traces collected from real website use (Lù et al., 2024); and AgentTrek explores trajectory synthesis from web tutorials as another route to scalable agent data (Xu et al., 2025). Taken together, these traces contain practical strategies for decomposing work, locating relevant pages, selecting candidate elements, recovering from mistakes, and recognizing when a subtask has been completed.

However, raw browser traces are a poor unit of reusable behavioral supervision. They are long, noisy, redundant, and tightly coupled to incidental page states. Low-level action targets depend on coordinates, DOM structures, login states, page versions, and website-specific implementations, so direct cloning of coordinates, element identifiers, or next-step actions can easily learn brittle page-specific patterns rather than transferable workflow competence. WebAgent explicitly combines planning, long-context understanding, and program synthesis to handle real-world web interaction beyond flat action imitation (Gur et al., 2024). SeeAct shows that grounding is itself a central bottleneck for vision-language web agents (Zheng et al., 2024), while WebVoyager studies end-to-end multimodal web agents under realistic browsing conditions (He et al., 2024a). The same issue appears beyond browsers: OSWorld highlights how open-ended computer-use tasks require robust observation

and action abstractions rather than narrow UI-state memorization (Xie et al., 2024).

We argue that the reusable unit of browser behavior is not an individual click, nor even an entire task trajectory, but a transferable natural-language skill. A skill describes a reusable subtask strategy: its goal, applicability conditions, key observations, element-selection principles, action templates, and common failure-recovery procedures. This view builds on evidence that language agents can benefit from persistent verbal experience. Reflexion stores textual feedback to improve later decisions (Shinn et al., 2023), ExpeL extracts reusable lessons from prior trials (Zhao et al., 2024), and Voyager maintains an expanding skill library for open-ended embodied exploration (Wang et al., 2023). Recent work on inducing programmatic skills further suggests that agent behavior can be represented and reused above the level of raw action sequences (Wang et al., 2025b). In our setting, skill summarization turns noisy low-level browser traces into higher-level semantic supervision, making behavior cloning less dependent on fragile page-specific actions.

This paper proposes a scalable framework for converting browser interaction traces into reusable natural-language skills. The framework cleans and normalizes trajectories, segments them into successful subtrajectories and failure-recovery episodes, applies distributed skill summarization to extract candidate skills, and then clusters, deduplicates, filters, and indexes them into a skill library. At inference time, a browser agent retrieves and composes relevant skills, then grounds skill-level intent into semantic element selection and executable browser actions. The resulting formulation is complementary to modular agent-training frameworks such as Agent Lumos (Yin et al., 2024) and to standardized web-agent environments such as BrowserGym (Chezelles et al., 2025).

We evaluate whether this skill-centric behavior cloning paradigm improves browser-agent behavior across diverse tasks. Our evaluation considers not only next-action prediction, but also end-to-end task success, recovery from failures, skill reuse, cross-site transfer, and robustness to page perturbations.

Contributions. This paper makes the following contributions:

- We observe that skill summarization can effectively extract high-level semantic structure from low-level browser demonstrations, reducing reliance on brittle coordinate- or element-level behavior cloning, and we verify that these summaries improve agent performance.
- We propose a scalable trace-to-skill framework that converts large-scale browser interaction trajectories into a retrievable, composable, and executable natural-language skill library.
- We validate the framework across a diverse set of browser tasks, measuring next-action prediction, end-to-end task success, failure recovery, skill reuse, cross-site generalization, and robustness to page perturbations.

2 Related Work

Web and computer-use agents. Research on browser agents has progressed along three coupled fronts: environments, benchmarks, and end-to-end policies. Early work established grounded language-agent interaction in controlled settings—WebShop in a simulated storefront (Yao et al., 2023) and Mind2Web over real websites (Deng et al., 2023)—while WebArena and VisualWebArena made evaluation realistic by demanding long-horizon interaction, multimodal perception, element grounding, and programmatic success checks (Zhou et al., 2024; Koh et al., 2024); WorkArena and BrowserGym extended this toward enterprise knowledge work and a common research infrastructure (Drouin et al., 2024; Boisvert et al., 2025; Chezelles et al., 2025). A parallel line strengthens the policy itself: WebVoyager and OpenWebVoyager study multimodal agents under realistic browsing (He et al., 2024a,b), SeeAct isolates grounding as the central bottleneck for vision-language agents (Zheng et al., 2024), and WebAgent couples planning, long-context reading, and program synthesis (Gur et al., 2024). Beyond the browser, OSWorld generalizes the problem to open-ended computer use and argues for robust observation–action abstractions over narrow UI-state memorization (Xie et al., 2024), and large-scale efforts such as ScaleCUA and OpenCUA characterize how competence scales with data and supervision (Liu et al., 2026; Wang et al., 2025a). More recent systems push capability through reinforcement learning and information-seeking objectives (Li et al., 2026; Wu et al., 2025; Vattikonda et al., 2025), process reward modeling (Chae et al., 2025), structured or hierarchical exploration (Yang et al., 2025; Gandhi and Neubig,

2026; Shen et al., 2025), and tool or context abstractions (Prabhu et al., 2026; Ye et al., 2026). The common thread is that browser competence improves with scale, abstraction, and richer interaction. Our contribution is orthogonal to all three fronts: rather than proposing new environments, rewards, or training objectives, we treat the trajectories already produced by ordinary internet users as a scalable supervision signal and ask how to convert them into reusable procedural knowledge.

Learning from web demonstrations. A substantial body of work uses human or agent trajectories as supervision. Mind2Web shows that demonstrations over real websites can supervise action grounding and next-step prediction (Deng et al., 2023); WebLINX collects multi-turn, dialogue-conditioned navigation traces from real-world use (Lù et al., 2024); and AgentTrek synthesizes trajectories from web tutorials as an alternative route to scalable data (Xu et al., 2025). Such traces implicitly encode how people decompose work, locate pages, select elements, recover from mistakes, and recognize completion. The difficulty is one of representation: raw traces are long, noisy, and tightly coupled to incidental page state, so cloning coordinates, selectors, or next-step actions directly tends to capture brittle, page-specific correlations rather than transferable competence. We share the goal of exploiting demonstrations at scale but take the reusable unit of supervision to be a transferable natural-language *skill* rather than a single action or a full trajectory, and we explicitly abstract away coordinates and selectors during distillation so that the guidance survives changes in page version and authentication state.

Skills, experience, and distributed memory. Our formulation builds on evidence that language agents benefit from persistent verbal experience. Reflexion converts outcome feedback into textual self-notes that improve later attempts (Shinn et al., 2023), ExpeL extracts reusable lessons across trials (Zhao et al., 2024), and Voyager maintains an expanding library of executable skills for open-ended embodied exploration (Wang et al., 2023). Related work induces programmatic skills that represent behavior above raw action sequences (Wang et al., 2025b), while modular frameworks such as Agent Lumos decouple planning, grounding, and execution into reusable components (Yin et al., 2024). A complementary line studies how experience scales when it is distributed: federated and distributed lifelong learning share knowledge across many private, non-IID clients or agents (McMahan et al., 2017; Rostami et al., 2018); LLM memory sharing pools useful experiences across agents (Gao and Zhang, 2024); and robotics efforts such as Open X-Embodiment show that heterogeneous demonstrations across platforms can support transfer (Open X-Embodiment Collaboration et al., 2024). We differ in source and representation: we distill *human* browser trajectories from many users, websites, and tasks into an auditable skill graph, so many trajectories can support one skill, one trajectory can yield several skills, and retrieved skills can be composed for new browser tasks.

3 Method

BROWSERBC learns reusable browser skills from interaction trajectories by lifting behavior cloning from low-level action imitation to graph-grounded skill distillation. Instead of treating demonstrations as sequences of coordinates, selectors, or atomic action labels, BROWSERBC regards them as procedural evidence of transferable browser behavior. The method first converts heterogeneous browser logs into trajectory graphs, then consolidates recurring transition patterns into a global skill graph, and finally distills each graph-supported skill into a natural-language procedure that can be retrieved, composed, and executed by a browser agent at inference time.

This design separates persistent behavioral knowledge from transient browser implementation details. Demonstrations provide procedural guidance about how task progress is achieved, while concrete element grounding remains conditioned on the live browser state. As a result, prior trajectories can guide future behavior without requiring the agent to reproduce brittle, page-specific action traces.

3.1 Problem Formulation

A browser trajectory is denoted as

$$\tau = (x, e_1, \dots, e_T, y),$$

where x is a natural-language task instruction, e_t is the t -th browser event, and y is the observed task outcome when available. Each event records the browser context together with the user action and any available execution feedback. A standard behavior cloning objective learns a policy

$$\pi(a_t \mid x, o_{\leq t}),$$

where a_t is a low-level browser action and $o_{\leq t}$ is the observation history. In realistic web environments, however, the same semantic operation may be realized through different layouts, interaction mechanisms, and implementation details. Directly cloning action labels or selectors therefore risks capturing surface-level correlations rather than transferable browser competence.

We instead formulate behavior cloning over a library of skills

$$\mathcal{L} = \{s_i\}_{i=1}^N,$$

where each skill s_i is a natural-language procedure induced from demonstrated state transitions. A skill specifies the conditions under which it applies, the strategy by which it advances the task, and the evidence by which its success or failure can be judged. Given a new task x^* and an optional current browser state v^* , the agent retrieves a subset of relevant skills

$$\mathcal{R}(x^*, v^*, \mathcal{L}) \subset \mathcal{L},$$

and uses them as procedural context for grounding actions in the current page. Under this formulation, demonstrations supervise the agent through reusable strategies and completion criteria rather than through one-step action imitation.

3.2 Trajectory Graph Construction

We first normalize heterogeneous browser logs into a common event representation and remove interactions that do not correspond to task-relevant state change. The cleaned trajectory is then partitioned around semantic transition boundaries rather than raw action boundaries. This produces subtrajectories that reflect functional changes in the task state, allowing the graph to capture how the user moves from one meaningful browser situation to another.

Each trajectory is represented as a directed graph

$$G_\tau = (V_\tau, E_\tau).$$

A node $v \in V_\tau$ denotes an abstract browser state. Rather than preserving the full page surface, the node summarizes the observable semantics of the browser context, including both the page-level situation and the task-relevant affordances available to the agent. An edge $e = (v_i, v_j) \in E_\tau$ denotes a demonstrated transition between two states and stores the corresponding action summary, task context, outcome, and transition role. We assign typed roles to edges, including progress, verification, recovery, fallback, failure, and loop transitions. These roles distinguish semantically different uses of similar actions; for example, submitting a form may complete a task, trigger validation errors, or serve as part of a recovery routine.

3.3 Skill Graph Induction

Across trajectories, BROWSERBC aggregates related transition edges into a global skill graph

$$G_s = (S, E_s, \Phi),$$

where S is the set of skill nodes, E_s contains typed relations between skills, and Φ maps each skill to its supporting trajectory evidence. A skill node is grounded not only in the textual description of actions, but also in the abstract states, transition roles, outcomes, and graph context from which it is induced. Consequently, two transitions are merged only when their procedural meaning is compatible: they should apply under comparable conditions, induce comparable state changes, and exhibit consistent empirical behavior.

The edges in E_s organize skills according to their structural dependencies. They capture whether one skill tends to enable another, whether a procedure should be understood as a specialization of a broader pattern, or whether two procedures are incompatible in the same state. This graph structure enables dependency-aware retrieval and multi-skill composition, rather than treating the skill library as a flat collection of text entries.

3.4 Distributed Skill Distillation

For each skill node $s \in S$, let

$$\Phi(s) = (v_i, e_{ij}, v_j)$$

denote its supporting state transitions. A distillation function D converts this graph-grounded evidence into an agent-readable procedure:

$$\hat{s} = D(\Phi(s), c_s),$$

where c_s includes contextual metadata such as domain, capability label, transition role, outcome statistics, and local graph neighborhood. The distillation model is instructed to remove session-specific artifacts, including coordinates, selectors, transient page values, and private user content, while preserving the reusable procedural structure.

Each distilled skill is stored as a structured skill card containing its intent, scope, observable preconditions, execution steps, progress milestones, terminal evidence, common failure modes, recovery strategies, negative constraints, and provenance metadata. Provenance links each skill to the trajectories that support it, making the library auditable and allowing future evidence to update or refine existing skills.

To scale skill construction, trajectory processing and local skill distillation are distributed across data shards. Each worker builds local trajectory graphs and proposes candidate skills from its assigned demonstrations. A global aggregation stage then aligns, deduplicates, merges, or specializes candidates according to their preconditions, effects, outcomes, and graph neighborhoods. When new trajectories are collected, only the affected skill nodes and adjacent graph regions need to be updated, enabling incremental library growth.

3.5 Skill Retrieval and Execution

At inference time, the agent retrieves skills conditioned on the task instruction and the current browser state. Given a task x^* and an inferred state node v^* , retrieval scores skills using semantic relevance, precondition compatibility, supported domains, terminal conditions, evidence statistics, and graph connectivity. When multi-step execution is required, the agent searches for a skill path

$$P^* = \arg \min_P C(P \mid x^*, v^*, G_s),$$

where the path cost C penalizes low-confidence transitions, historically failure-prone edges, loops, and conflicts, while rewarding semantic relevance and compositional compatibility.

The retrieved skills are inserted into the agent context as procedural guidance. The agent still observes the live browser page and grounds each step into executable actions at runtime. The skills constrain this grounding process by specifying what to look for, which task values to preserve, how to recognize progress, when to stop, and how to recover from common failures. In this way, BROWSERBC transfers behavior from prior demonstrations without copying brittle low-level action traces.

3.6 Evidence Granularity

BROWSERBC supports different evidence regimes under a unified graph formulation. When evidence is sparse, a single successful trajectory can be distilled into a case-specific skill that preserves the task structure observed in that demonstration. As additional attempts become available, successes and failures are consolidated into an evolving skill whose execution guidance becomes more robust. In the main setting considered in this work, large collections of trajectories are mapped into a distributed skill graph, allowing reusable procedures to be shared, composed, and updated across users, websites, and task families.

Overall, BROWSERBC reframes browser behavior cloning as graph-grounded skill distillation. The trajectory graph preserves demonstrated state transitions, the skill graph organizes these transitions into typed and composable procedural units, and the distilled skills guide future agents through semantic constraints rather than brittle low-level action imitation.

4 Experiments

4.1 ClawBench

4.2 WebArena (@zbs)

We evaluate whether distilling human browser trajectories into reusable natural-language skills improves end-to-end task success for a strong browser agent on WebArena. Our central question is direct: holding the agent, environment, and evaluation fixed, does equipping the agent with BROWSERBC skills raise its success rate, and on what kinds of tasks? We first describe the evaluation protocol (§4.2.1), then report the main skill-on / skill-off comparison on WebArena (§4.2.2), analyze where skills help and where they do not (§4.2.3), and finally summarize the integrity safeguards that make the comparison trustworthy (§4.2.5).

4.2.1 Setup

Benchmark. We evaluate on the hard split of WebArena (Zhou et al., 2024), comprising 258 human-verified tasks across five self-hosted sites (GitLab, an e-commerce storefront and its admin panel, a Reddit-style forum, and an OpenStreetMap-based map service). Each task is scored by its original programmatic checker, which inspects either the agent’s final answer (*string-match*: exact, fuzzy, or required-substring), the resulting page URL (*url-match*), or the live page state via DOM/JavaScript locators (*program-html*). We report task success rate (SR), the fraction of tasks whose checker returns full credit. To make the analysis interpretable, we group tasks into three categories: information-seeking *read-only* tasks (85), state-changing *mutation* tasks (154), and *map* routing/geocoding tasks (19).

Agent and harness. Both conditions use the same browser agent: a Claude model driven through an agentic CLI loop, acting on the live page through a Playwright-based Model-Context-Protocol (MCP) browser that exposes both a rendered *screenshot* (vision) and a structured *DOM/accessibility snapshot* at every step, with element grounding and actions issued back over CDP. This multimodal, official-aligned harness is held identical across conditions; the only variable is whether BROWSERBC skills are injected into the agent context. Each task is given the same step and wall-clock budget in both conditions.

Conditions. We compare two configurations of the identical agent: **(i) skill-off (base)**, the agent with no external skills; and **(ii) skill-on (BROWSERBC)**, the same agent additionally given the distilled natural-language skill for the task family, retrieved from the human-trajectory skill library. The skills are produced by the pipeline of §?: human demonstrations are normalized, segmented, and distilled into procedural skills describing applicability conditions, a generalized execution procedure, progress milestones and terminal evidence, and common failure-recovery steps. Crucially, skills describe *how to perform* a task family at the level of page semantics and element-selection principles; they contain no benchmark answers and no checker-specific values (§4.2.5).

4.2.2 Main Result: Skills Improve Browser-Agent Success

Table 1 reports the comparison on WebArena-Hard: the identical agent is run on all 258 tasks with skills off and with the BROWSERBC skill library, and we report success rate. Equipping the agent with distilled human-trajectory skills raises overall success from 60.5% to 81.4%, a +20.9 point absolute improvement (+34.5% relative) while leaving the agent, environment, evaluation, and budget unchanged. The gain is large across every task category and concentrates where procedural knowledge matters most. We report BROWSERBC as the skill-augmented agent that retrieves and applies skills; a strict ablation that instead forces the agent to follow every retrieved skill verbatim—even when live page evidence contradicts it—reaches 77.5%, so part of the gain comes from the agent applying skills selectively rather than blindly, consistent with the confidence-aware design of §?.

4.2.3 Analysis

Where do skills help most? The largest improvement is on read-only, information-seeking tasks (+29.4 points). These tasks fail in the base agent not because the answer is unreachable, but because the agent searches the

Table 1 Task success rate (SR) on WebArena-Hard (258 tasks) for the identical browser agent with skills off (base) and with the BROWSERBC skill library. Skills are distilled from human browser trajectories and contain procedural guidance only. Δ is the absolute improvement in percentage points.

Category	# Tasks	Skill-off (base)	BROWSERBC	Δ
Read-only	85	56.5	85.9	+29.4
Mutation	154	62.3	78.6	+16.3
Map	19	63.2	84.2	+21.0
All	258	60.5	81.4	+20.9

wrong way, stops early, or reads an incomplete slice of a long result list or table. The distilled skills encode exactly the transferable procedure a human used—which page and sort order to open, which products or rows are genuinely relevant, and how to read minimum and maximum values—turning a fragile search into a reliable one. Map tasks improve comparably (+21.0 points): the skills supply the correct routing/geocoding procedure (which service path to query and how to land on the destination’s map page) that the base agent often reaches only unreliably. Mutation tasks improve substantially as well (+16.3 points): skills reliably fix *navigation and progress recognition* (reaching the correct form, knowing which terminal page constitutes completion), with the residual failures dominated by genuine multi-field execution precision, where the agent must set many interdependent fields correctly in a single form.

What remains hard. A manual audit of the residual failures shows they are concentrated in *agent execution limits* rather than missing knowledge: multi-field admin-panel forms where one of several required attributes is omitted, search aggregations where one item of a list is dropped, and tasks requiring an exact target among several plausible candidates. For these, providing an even more detailed skill does not help—the procedure is already correct, but the agent’s step-by-step execution is not perfectly reliable. This delineates a clean boundary: BROWSERBC closes the gap caused by *not knowing the procedure*, and the remaining headroom is *procedure-following fidelity*, which is a property of the underlying agent rather than of the skill library.

Skills are not universally beneficial. The improvement is a strong net effect, not a uniform one. On a small fraction of tasks (10 of 258, 3.9%), blindly following the retrieved skill makes the agent fail a task it solves on its own, and these regressions persist across repeated runs rather than being sampling noise. They occur when an over-specific or imperfectly-distilled skill anchors the agent to one strategy and discourages the simpler exploration that would otherwise have succeeded. This is exactly the gap between the strict always-follow ablation (77.5%) and BROWSERBC (81.4%): allowing the agent to apply a skill selectively—and to defer to live page evidence when it conflicts with the skill—recovers these regressions. It exposes an honest property of behavior cloning from summaries (a skill encodes a prior, and an over-confident prior can mislead an otherwise-capable agent) and motivates the confidence-aware, evidence-weighted retrieval of §??, so that the agent trusts a retrieved skill in proportion to its demonstrated reliability rather than following it unconditionally.

4.2.4 Failure Case Study

To make the residual headroom concrete, we group the surviving failures of BROWSERBC into four recurring patterns and give a representative case for each. (i) *Multi-field execution*. Tasks that require setting several interdependent fields in one submission—creating a GitLab issue with both an assignee and a due date, or an admin-panel product with the correct attribute set, price, quantity, size, and category—fail when the agent populates most fields correctly but silently omits one. The skill already enumerates every field; the breakdown is in reliably executing a long form, not in knowing what to enter. (ii) *Ambiguous target selection*. Tasks whose checker requires one *specific* object among several plausible ones—posting a comment that must link a particular repository URL when the user owns many similar repositories—fail when the agent acts on a near-duplicate. Here the demonstration itself is genuinely under-determined, so the distilled skill cannot disambiguate further. (iii) *Long-horizon budget exhaustion*. Tasks that chain a long information-gathering phase into a final action—reading a dozen product reviews and then composing a forum post—occasionally

exhaust the step budget mid-task and terminate on an intermediate page, so the final-state checker sees the wrong URL. (iv) *Generation-limit runaway*. A small number of tasks trigger over-long internal reasoning that hits the model’s output ceiling before the action completes; these are properties of the backbone model, not of the skill. Across all four patterns the skill content is correct and applicable—the limit is the agent’s execution fidelity, consistent with the boundary identified in §4.2.3. A separate, smaller failure mode is the skill-*induced* regression analyzed above, where an over-specific prior steers an otherwise-capable agent away from the simpler successful path.

4.2.5 Evaluation Integrity

Because skills are natural-language text inserted into the agent context, a naive distillation can leak benchmark-specific answers and inflate results. We take two safeguards. First, distilled skills are constrained to *procedural* content—applicability conditions, navigation, element-selection principles, milestones, and recovery—and we audit the library to remove any checker-targeting content (e.g., copied required substrings, hard-coded field values, or instructions phrased against the grader); the reported numbers are measured *after* this de-leakage pass, so improvements reflect transferable procedure rather than memorized answers. Second, during analysis we identified a small number of genuine *checker* defects in the benchmark itself that penalize correct agent behavior—internally inconsistent task/checker dates, host-alias mismatches between reference URLs and the self-hosted deployment (e.g. `www.reddit.com` vs. the local host), whitespace-sensitive name matching, and stale hard-coded “newest/most-recent” item identifiers that no longer match the deployed data. We correct these defects with normalizations that are additive (they can only turn a previously-missed correct answer into a hit, never the reverse) or anchored to verifiable ground truth, and we apply the identical fixes to *both* conditions. These corrections are independent of the skill library and equally benefit the skill-off baseline. Together, these safeguards ensure the skill-on / skill-off gap measures the value of distilled human procedure, not answer leakage or benchmark artifacts.

4.3 OS World

4.4 9 Website Demo

5 Conclusion

This draft sets up a paper on scalable behaviour cloning for browser agents via distributed skill summarization. The main manuscript should develop the distributed trajectory pipeline, skill extraction procedure, skill consolidation layer, retrieval interface, and evaluation protocol into a complete empirical study. The current template is intentionally clean: old paper content has been removed, while the original formatting system is preserved for the new paper.

[TODO] new cases in appendix

6 Contributions and Acknowledgements

The contributors' names are sorted in alphabetical order of the last name.

Core Contributors

Yuzhao Peng, Houde Qian, Kaisen Yang, Boshi Zhang, Youjie Zheng

References

- Léo Boisvert, Megh Thakkar, Maxime Gasse, Massimo Caccia, Thibault Le Sellier De Chezelles, Quentin Cappart, Nicolas Chapados, Alexandre Lacoste, and Alexandre Drouin. Workarena++: Towards compositional planning and reasoning-based common knowledge work tasks, 2025. <https://arxiv.org/abs/2407.05291>.
- Hyungjoo Chae, Sunghwan Kim, Junhee Cho, et al. Web-shepherd: Advancing PRMs for reinforcing web agents. In *Advances in Neural Information Processing Systems*, 2025. <https://openreview.net/forum?id=G2kMroO9UV>.
- Thibault Le Sellier De Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omid Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Quentin Cappart, Graham Neubig, Ruslan Salakhutdinov, Nicolas Chapados, and Alexandre Lacoste. The browsergym ecosystem for web agent research, 2025. <https://arxiv.org/abs/2412.05467>.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web, 2023. <https://arxiv.org/abs/2306.06070>.
- Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. Workarena: How capable are web agents at solving common knowledge work tasks?, 2024. <https://arxiv.org/abs/2403.07718>.
- Apurva Gandhi and Graham Neubig. Go-browse: Training web agents with structured exploration. In *International Conference on Learning Representations*, 2026. <https://openreview.net/forum?id=IpzRWE52yw>.
- Hang Gao and Yongfeng Zhang. Memory sharing for large language model based agents, 2024. <https://arxiv.org/abs/2404.09982>.
- Izzeddin Gur, Hiroki Furuta, Austin Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. A real-world webagent with planning, long context understanding, and program synthesis, 2024. <https://arxiv.org/abs/2307.12856>.
- Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models, 2024a. <https://arxiv.org/abs/2401.13919>.
- Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Hongming Zhang, Tianqing Fang, Zhenzhong Lan, and Dong Yu. Openwebvoyager: Building multimodal web agents via iterative real-world exploration, feedback and optimization, 2024b. <https://arxiv.org/abs/2410.19609>.
- Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks, 2024. <https://arxiv.org/abs/2401.13649>.
- Kuan Li, Zhongwang Zhang, Huifeng Yin, et al. Websailor-v2: Bridging the chasm to proprietary agents via synthetic data and scalable reinforcement learning. In *International Conference on Learning Representations*, 2026. <https://openreview.net/forum?id=HuP16O5SJf>.
- Zhaoyang Liu, JingJing Xie, Zichen Ding, et al. Scalecua: Scaling open-source computer use agents with cross-platform data. In *International Conference on Learning Representations*, 2026. <https://openreview.net/forum?id=yBFUqdJFZn>.
- Xing Han Lù, Zdeněk Kasner, and Siva Reddy. Weblinx: Real-world website navigation with multi-turn dialogue, 2024. <https://arxiv.org/abs/2402.05930>.
- H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, pages 1273–1282, 2017. <https://proceedings.mlr.press/v54/mcmahan17a.html>.
- Open X-Embodiment Collaboration, Abby O'Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, et al. Open X-Embodiment: Robotic learning datasets and RT-X models. In *2024 IEEE International Conference on Robotics and Automation*, pages 6892–6903, 2024. doi: 10.1109/ICRA57147.2024.10611477. <https://arxiv.org/abs/2310.08864>.
- Viraj Prabhu, Yutong Dai, Matthew Fernandez, et al. WALT: Web agents that learn tools. In *International Conference on Learning Representations*, 2026. <https://openreview.net/forum?id=cgIDqcJcoI>.

- Mohammad Rostami, Soheil Kolouri, Kyungnam Kim, and Eric Eaton. Multi-agent distributed lifelong learning for collective knowledge acquisition. In *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*, pages 712–720, 2018. <https://www.ifaamas.org/Proceedings/aamas2018/pdfs/p712.pdf>.
- Junhong Shen, Hao Bai, Lunjun Zhang, et al. Thinking vs. doing: Improving agent reasoning by scaling test-time interaction. In *Advances in Neural Information Processing Systems*, 2025. <https://openreview.net/forum?id=un1TRwNgiv>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. <https://arxiv.org/abs/2303.11366>.
- Dheeraj Vattikonda, Santhoshi Ravichandran, Emiliano Penaloza, et al. How to train your LLM web agent: A statistical diagnosis. In *Advances in Neural Information Processing Systems*, 2025. <https://openreview.net/forum?id=67xkPEM3bZ>.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. <https://arxiv.org/abs/2305.16291>.
- Xinyuan Wang, Bowen Wang, Dunjie Lu, et al. Opencua: Open foundations for computer-use agents. In *Advances in Neural Information Processing Systems*, 2025a. <https://openreview.net/forum?id=6iRZvJiC9Q>.
- Zora Zhiruo Wang, Apurva Gandhi, Graham Neubig, and Daniel Fried. Inducing programmatic skills for agentic tasks, 2025b. <https://arxiv.org/abs/2504.06821>.
- Jialong Wu, Baixuan Li, Runnan Fang, et al. Webdancer: Towards autonomous information seeking agency. In *Advances in Neural Information Processing Systems*, 2025. <https://openreview.net/forum?id=quJdphBcdP>.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Oworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. <https://arxiv.org/abs/2404.07972>.
- Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, and Tao Yu. Agenttrek: Agent trajectory synthesis via guiding replay with web tutorials, 2025. <https://arxiv.org/abs/2412.09605>.
- Qianlan Yang, Xiangjun Wang, Danielle Perszyk, and Yu-Xiong Wang. Self-guided hierarchical exploration for generalist foundation model web agents. In *Advances in Neural Information Processing Systems*, 2025. <https://openreview.net/forum?id=9twwDW60Bw>.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents, 2023. <https://arxiv.org/abs/2207.01206>.
- Rui Ye, Zhongwang Zhang, Kuan Li, et al. Agentfold: Long-horizon web agents with proactive context folding. In *International Conference on Learning Representations*, 2026. <https://openreview.net/forum?id=IuZoTgsUws>.
- Da Yin, Faeze Brahman, Abhilasha Ravichander, Khyathi Chandu, Kai-Wei Chang, Yejin Choi, and Bill Yuchen Lin. Agent lumos: Unified and modular training for open-source language agents, 2024. <https://arxiv.org/abs/2311.05657>.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, 2024. <https://arxiv.org/abs/2308.10144>.
- Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. Gpt-4v(ision) is a generalist web agent, if grounded, 2024. <https://arxiv.org/abs/2401.01614>.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents, 2024. <https://arxiv.org/abs/2307.13854>.

A Appendix

A.1 Implementation Details

Add skill-summary prompt format, distributed collection protocol, deduplication/consolidation details, browser runtime details, and retrieval configuration here.

A.2 Additional Results

Add full tables, per-task breakdowns, and qualitative examples here.